

Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.callbacks import EarlyStopping

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    roc_curve,
    auc,
    roc_auc_score
)

from sklearn.preprocessing import label_binarize
```

Load Dataset

```
# Load Fashion MNIST
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0
```

Build Model

```
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Task 1: Train Model and Plot Loss Curves

```
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_test, y_test),
    epochs=10,
    batch_size=64
)

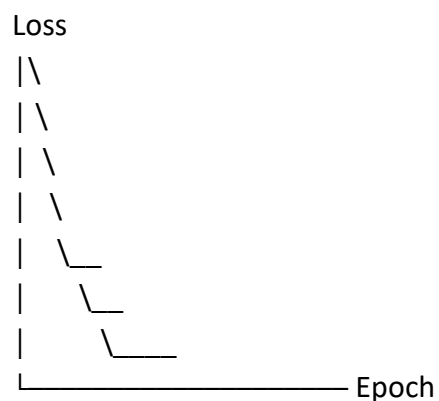
# Plot Loss
plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label="Training Loss")
plt.plot(history.history['val_loss'], label="Validation Loss")

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.show()
```

Expected Graph



Blue → Training Loss

Orange → Validation Loss

Task 2: Intentionally Overfit

Train only on **500 samples**.

```
x_small = x_train[:500]
```

```
y_small = y_train[:500]
```

```
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(256, activation='relu'),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    x_small,
    y_small,
    validation_data=(x_test,y_test),
    epochs=30
)
```

```
plt.plot(history.history['loss'], label="Training")
plt.plot(history.history['val_loss'], label="Validation")
plt.legend()
plt.show()
```

Two Signs of Overfitting

1. **Training loss keeps decreasing**, while **validation loss starts increasing** after a few epochs.

2. **The gap between training and validation loss widens**, showing the model is memorizing the training data instead of generalizing.
-

Task 3: EarlyStopping

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    x_train,
    y_train,
    validation_data=(x_test,y_test),
    epochs=30,
    callbacks=[early_stop]
)

print("Training stopped after:",
      len(history.history['loss']),
      "epochs")
```

Example Output

Training stopped after: 12 epochs

The exact number of epochs depends on how quickly the validation loss stops improving.

Task 4: Accuracy, Precision, Recall

```
# Predict probabilities
y_prob = model.predict(x_test)

# Convert to class labels
y_pred = np.argmax(y_prob, axis=1)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
```

```
        average='weighted'
    )

recall = recall_score(
    y_test,
    y_pred,
    average='weighted'
)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall  :", recall)
```

Example Output

Accuracy : 0.89

Precision : 0.89

Recall : 0.89

Task 5: ROC Curve and AUC

```
# One-hot encode labels
y_test_bin = label_binarize(y_test, classes=range(10))

# Predict probabilities
y_score = model.predict(x_test)

# ROC AUC
auc_score = roc_auc_score(
    y_test_bin,
    y_score,
    multi_class='ovr'
)

print("AUC Score:", auc_score)

# ROC Curve for Class 0
fpr, tpr, _ = roc_curve(
    y_test_bin[:,0],
```

```
y_score[:,0]
)

roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7,6))

plt.plot(
    fpr,
    tpr,
    label=f"AUC = {roc_auc:.3f}"
)

plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve (Class 0)")
plt.legend()

plt.grid(True)

plt.show()
```

Example Output

AUC Score: 0.98

Explanation

1. Loss Curves

- **Training Loss:** Error on the training dataset.
- **Validation Loss:** Error on unseen validation data.

A good model has both losses decreasing together and remaining close.

2. Overfitting

Overfitting occurs when the model learns the training data too well but performs poorly on new data.

Signs:

- Training loss continues to decrease.
 - Validation loss starts increasing.
 - Large gap between training and validation curves.
-

3. EarlyStopping

EarlyStopping monitors validation loss.

```
EarlyStopping(  
    monitor='val_loss',  
    patience=3,  
    restore_best_weights=True  
)
```

- Stops training if validation loss does not improve for **3 consecutive epochs**.
 - Restores the weights from the best-performing epoch.
-

4. Evaluation Metrics

- **Accuracy:** Percentage of correct predictions.
- **Precision:** Of the predicted positive classes, how many are correct.
- **Recall:** Of the actual positive classes, how many are correctly identified.

For multi-class classification, using average='weighted' accounts for class imbalance.

5. ROC Curve & AUC

- **ROC Curve:** Plots the True Positive Rate against the False Positive Rate.
- **AUC (Area Under the Curve):** Measures how well the model separates classes.
 - **1.0:** Perfect classifier.
 - **0.5:** Random guessing.
 - **0.9+:** Excellent classifier.

For Fashion MNIST (10 classes), `roc_auc_score(..., multi_class='ovr')` computes a one-vs-rest ROC-AUC score suitable for multiclass problems.